Scenario:

Tech University, a software institute, requires a robust web API application to efficiently manage student course enrollments and instructor course assignments. This solution aims to streamline administrative tasks, enhance data accuracy, and improve overall operational efficiency.

Help them implement the required services by creating a web API application using the **Web API Core** with an **Entity Framework Core** and making them work more efficiently.

Functionalities:

1. **Course** - CRUD operations on Course info and related business logic.
2. **Instructor** - CRUD operations on Instructor's info and related business logic.
3. **Student** - CRUD operations on Student's info and related business logic.

Requirements:

1. MODELS

The below-given classes and properties and data annotations are already provided in the code snippet

1. **Course.cs**
2. **Enrollment.cs**
3. **Instructor.cs**
4. **InstructorCourse.cs**
5. **Student.cs**

2. Data :- Class **UniversityContext.cs** is already provided in the code snippet

3. CONTROLLERS

In class **CourseController.cs,** implement the below-mentioned actions.

Service	**Http Type & Return Type**	**Functionality**	**Repository (Check CourseRepository Section)**
api/[controller]/ UpdateCourse	PUT Method & IActionResult	If the result is **true**, it should return an HTTP response with "**Ok**" status. Otherwise, return "**BadRequest**". **Parameter:** Get a JSON representation of a **"Course "** object from the request body.	It gets the result from a course repository by calling the UpdateCourse() method.
api/[controller]/ WithEnrollmentsAboveGrade /{grade}	GET Method & IActionResult	If the course is found, it should return an HTTP response with the retrieved courses in the "**OK**" status. Otherwise, return "**Not Found**" with the message **No Records Found.** **Parameter:** Pass the **grade** as an **int**.	It retrieves the course details from a course repository by calling the GetCoursesWithEnrollmentsAboveGrade() method.
api/[controller]/ ByInstructorName/ {instructorName}	GET Method & IActionResult	If the course is found, it should return an HTTP response with the retrieved courses in the "**OK**" status. Otherwise, return "**Not Found**" with the message **No Records Found**. **Parameter :** Pass the **instructorName** as an **string**.	It retrieves the course details from a course repository by calling the GetCoursesByInstructorName() method.

In class **InstructorController.cs,** implement the below-mentioned actions.

Service	**Http Type & Return Type**	**Functionality**	**Repository (Check InstructorRepository Section)**
api/[controller]/ AddInstructor	POST Method & IActionResult	If the result is **true**, it should return an HTTP response with	It gets the result from an instructor repository by calling the AddInstructor() method.

		"**Ok**" status. Otherwise, return "**BadRequest**". **Parameter :** Get a JSON representation of an **"Instructor"** object from the request body.	
api/[controller]/ WithCourseCountAbove/ {count}	GET Method & IActionResult	If the instructor is found, it should return an HTTP response with the retrieved instructors in the "**OK**" status. Otherwise, return "**Not Found**" with the message **No Records Found**. **Parameter :** Pass the **count** as an **int**.	It retrieves the instructor details from an instructor repository by calling the GetInstructorsWithCourseCountAbove() method
api/[controller]/ WithMostEnrollments	GET Method & IActionResult	If the instructor is found, it should return an HTTP response with the retrieved instructors in the "**OK**" status. Otherwise, return "**Not Found**" with the message **No Records Found**.	It retrieves the instructor details from an instructor repository by calling the GetInstructorsWithMostEnrollments() method

In class **StudentController.cs,** implement the below-mentioned actions.

Service	**Http Type & Return Type**	**Functionality**	**Repository (Check StudentRepository Section)**
api/[controller]/ DeleteStudent/{studentId}	DELETE Method & IActionResult	If the result is **true**, it should return an HTTP response with "**Ok**" status. Otherwise, return "**Not Found**" with the message **No Records Found**. **Parameter:** Pass the **studentId** as an **int**.	It gets the result from a student repository by calling the DeleteStudent() method.
api/[controller]/ ByCourseTitle /{courseTitle}	GET Method & IActionResult	If the student is found, It should return an HTTP response with the retrieved students in the "**OK**" status. Otherwise, return "**Not Found**" with the message **No Records Found**. **Parameter:** Pass the **courseTitle** as a **string**.	It retrieves the course details from a student repository by calling the GetStudentsByCourseTitle() method.

Note: In the controllers class, use the **dependency injection** concept to invoke repository class methods.

4. INTERFACES

In interface **ICourse.cs**, just declare the below methods,

Method Name
bool UpdateCourse(Course course)
IEnumerable<Course> GetCoursesWithEnrollmentsAboveGrade(int grade)
IEnumerable<Course> GetCoursesByInstructorName(string instructorName)

In interface **IInstructor.cs**, just declare the below methods,

Method Name
bool AddInstructor(Instructor instructor)
IEnumerable<Instructor> GetInstructorsWithCourseCountAbove(int count)
IEnumerable<Instructor> GetInstructorsWithMostEnrollments()

In interface **IStudent.cs**, just declare the below methods,

Method Name
bool DeleteStudent(int studentId)
IEnumerable<Student> GetStudentsByCourseTitle(string courseTitle)

5. REPOSITORIES

In class **CourseRepository.cs,** implement the below-mentioned methods and also implement the interface **ICourse**.

Method	Functionality
public bool UpdateCourse(Course course)	This method is used to update the course details in the **Courses** table. Check if the course Id is available in the table: if it's found, then update the **course title** and return **true**. Otherwise, return **false**.
public IEnumerable<Course> GetCoursesWithEnrollmentsAboveGrade(int grade)	This method retrieves a list of **courses** that have at least one **enrollment** with a **grade** higher than the specified value. It queries the database context to find courses meeting this condition and **returns** the matching courses as a list.
public IEnumerable<Course> GetCoursesByInstructorName(string instructorName)	This method retrieves a list of **courses** taught by an **instructor** with the specified name. This method returns an IEnumerable collection of courses associated with an instructor whose name matches the provided **instructorName**. It finds entries where the instructor's name matches the given name and then selects the corresponding **courses**. Store the matched courses and return it as a **list**.

In class **InstructorRepository.cs,** implement the below-mentioned methods and also implement the interface **IInstructor**.

Method	Functionality
public bool AddInstructor(Instructor instructor)	This method is used to add the instructor details to the **Instructors** table. If the instructor already exists in the table, then return **false**. Otherwise, add and return **true**.
public IEnumerable<Instructor> GetInstructorsWithCourseCountAbove(int count)	This method retrieves a list of **instructors** who teach more than a specified number of **courses**. This method returns an IEnumerable collection of instructors who are associated with more than a given number of **courses**. It filters the groups to include only those with a **count** of courses greater than the specified **count** and then selects the corresponding **instructors** from these groups. The resulting list of instructors is then **returned**.
public IEnumerable<Instructor> GetInstructorsWithMostEnrollments()	This method retrieves **instructors** who have the **highest** number of **enrollments** across all their courses. It calculates the enrollment count for each instructor by tallying all enrollments in the courses they teach. Then, it identifies the **maximum** enrollment count among all instructors. Finally, it filters out instructors whose enrollment count matches this **maximum**, **returning** them as a collection.

In class **StudentRepository.cs,** implement the below-mentioned methods and also implement the interface **IStudent**.

Method	Functionality
public bool DeleteStudent(int studentId)	This method is used to delete the student details by id from the **Students** table and return **true**. If the id is not found, then return **false**.
public IEnumerable<Student> GetStudentsByCourseTitle(string courseTitle)	This method retrieves students who are enrolled in a course with a specified **title**. It searches through the database for enrollments, filtering them based on the course title provided. Then, it selects the corresponding **students** from these enrollments and returns them as a **list.**

Note:

- Do not change or delete the class/method names or return types provided to you as part of the code skeleton.
- Use **Entity Framework Core** as the ORM, including necessary **migration comments**, to efficiently manage database interactions.
- Utilize **SQL Server Object Explorer** in Visual Studio to connect and manage the **LocalDB** instance for database development and exploration.

6. SAMPLE INPUT/OUTPUT

Note: The sample input and output images are provided for only some valid and invalid conditions, not all.

PUT /api/Course/UpdateCourse

Parameters Cancel

No parameters

Request body application/jso

```
{
  "courseId": 102,
  "title": "C++",
  "enrollments": [


  ],
  "instructorCourses": [

  ]
}
```

Request URL

```
https://localhost:7131/api/Course/UpdateCourse
```

Server response

Code	Details
200	Response headers

```
content-length: 0
date: Fri,24 May 2024 10:39:33 GMT
server: Kestrel
```

Responses

Code	Description
200	Success

GET /api/Course/WithEnrollmentsAboveGrade/{grade}

Parameters

Name	Description
grade * required integer($int32) *(path)*	90

Request URL

```
https://localhost:7131/api/Course/WithEnrollmentsAboveGrade/90
```

Server response

Code	Details
200	Response body

```
[
  {
    "courseId": 103,
    "title": "Python",
    "enrollments": null,
    "instructorCourses": null
  }
]
```

Response headers

```
content-type: application/json; charset=utf-8
date: Fri,24 May 2024 10:44:44 GMT
server: Kestrel
```

Responses

Code	Description
200	Success

GET /api/Course/ByInstructorName/{instructorName}

Parameters

Name	Description
instructorName * required string (path)	Lee

Request URL

```
https://localhost:7131/api/Course/ByInstructorName/Lee
```

Server response

Code	Details
200	Response body

```
[
  {
    "courseId": 102,
    "title": "C++",
    "enrollments": null,
    "instructorCourses": null
  }
]
```

Response headers

```
content-type: application/json; charset=utf-8
date: Fri,24 May 2024 10:46:04 GMT
server: Kestrel
```

Responses

Code	Description
200	Success

GET /api/Instructor/WithCourseCountAbove/{count}

Parameters

Name	Description
count * required integer($int32) (path)	2

Request URL

```
https://localhost:7131/api/Instructor/WithCourseCountAbove/2
```

Server response

Code	Details
200	Response body

```
[
  {
    "instructorId": 203,
    "name": "Williams",
    "instructorCourses": null
  }
]
```

Response headers

```
content-type: application/json; charset=utf-8
date: Fri,24 May 2024 10:49:15 GMT
server: Kestrel
```

Responses

Code	Description
200	Success

GET /api/Instructor/WithMostEnrollments

Parameters

No parameters

Execute | Clear

Responses

Curl

```
curl -X 'GET' \
  'https://localhost:7131/api/Instructor/WithMostEnrollments' \
  -H 'accept: */*'
```

Request URL

```
https://localhost:7131/api/Instructor/WithMostEnrollments
```

Server response

Request URL

```
https://localhost:7131/api/Instructor/WithMostEnrollments
```

Server response

Code	Details
200	Response body

```
[
  {
    "instructorId": 203,
    "name": "Williams",
    "instructorCourses": null
  }
]
```

Response headers

```
content-type: application/json; charset=utf-8
date: Fri,24 May 2024 10:50:19 GMT
server: Kestrel
```

GET /api/Student/ByCourseTitle/{courseTitle}

Parameters

Name	Description
courseTitle * required string *(path)*	C#

Request URL

```
https://localhost:7131/api/Student/ByCourseTitle/C%23
```

Server response

Code	Details
200	Response body

```
[
  {
    "studentId": 1,
    "name": "John Doe",
    "enrollments": null
  }
]
```

Response headers

```
content-type: application/json; charset=utf-8
date: Fri,24 May 2024 10:52:36 GMT
server: Kestrel
```

POST /api/Instructor/AddInstructor

Parameters

Cancel

No parameters

Request body

application/jsc

```
{
  "instructorId": 201,
  "name": "Kevin",
  "instructorCourses": [

  ]
}
```

Request URL

https://localhost:7131/api/Instructor/AddInstructor

Server response

Code	Details
400 *Undocumented*	Error: response status is 400

Response body

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "Bad Request",
  "status": 400,
  "traceId": "00-0097b88ebd40cfcc0cb7769fa3045a92-31a9ccfccd5db339-00"
}
```

Response headers

```
content-type: application/problem+json; charset=utf-8
date: Sat,25 May 2024 04:58:33 GMT
server: Kestrel
```

DELETE /api/Student/DeleteStudent/{studentId}

Parameters

Name	Description
studentId * required integer($int32) *(path)*	111

Request URL

https://localhost:7131/api/Student/DeleteStudent/111

Server response

Code	Details
404 *Undocumented*	Error: response status is 404

Response body

```
No Records Found
```

Response headers

```
content-type: text/plain; charset=utf-8
date: Mon,27 May 2024 13:40:10 GMT
server: Kestrel
```